Equipe 9 Engenharia de Software	**METODOLOGIA DE DESENVOLVIMENTO DE SOFTWARE** **Documento de Arquitetura de Software**

RollCore – RPG Skill Checker & Companion

Responsável pelo Projeto	Equipe de Desenvolvimento
Equipe 9 Disciplina de Análise e Projeto de Software	João Pedro Nunes Neto, Leonardo dos Santos Silva, Lucas Gabriel Pereira Guerra, Luis Felipe Nunes da Fonseca Figueiredo, Luiz Phillipe de Souza Santos

Objetivo deste Documento
Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada dessas decisões. Suas informações incluem a parte de hardware e software do sistema. O documento adota uma estrutura baseada na visão "4+1" de modelo de arquitetura.

Data	Versão	Demanda	Descrição	Autor
Abr/2026	1.0	Entrega 3	Versão inicial do Documento de Arquitetura.	Equipe 9

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema RollCore – RPG Skill Checker & Companion, usando diversas visões de arquitetura para representar diferentes aspectos do sistema. O objetivo é capturar e comunicar as decisões arquiteturais significativas tomadas em relação ao sistema, orientando a equipe de desenvolvimento, a equipe de QA e demais stakeholders.

O documento adota uma estrutura baseada na visão "4+1" de modelo de arquitetura [KRU41], complementada pelas seções de Dimensionamento, Performance e Qualidade do template DataSUS.

1.2 Escopo

Este Documento de Arquitetura de Software se aplica ao RollCore – RPG Skill Checker & Companion, desenvolvido pela Equipe 9 no âmbito da disciplina de Análise e Projeto de Software.

O RollCore é uma plataforma digital multiplataforma (web e, futuramente, mobile) voltada a jogadores e mestres de RPG de mesa. O sistema abrange:

- Criação e gestão de fichas de personagens D&D 5e com cálculo automático de atributos, modificadores, CA, HP e slots de magia.
- Rolagem de dados virtuais com suporte a fórmulas (NdX+M) e histórico de rolagens.
- Autenticação segura de usuários com JWT e persistência de dados na nuvem.
- Sessões online em tempo real com sala virtual compartilhada entre mestre e jogadores (Fase 2).
- Suporte a múltiplos sistemas de RPG, iniciando por D&D 5e e expandindo para Ordem Paranormal (Fase 3).

1.3 Definições, Acrônimos e Abreviações

Abreviação / Termo	Definição
D&D 5e	Dungeons & Dragons 5ª Edição – sistema de RPG de mesa da Wizards of the Coast, utilizado sob licença SRD Creative Commons.
SRD	System Reference Document – conteúdo de domínio público do D&D 5e sob licença Creative Commons CC BY 4.0.
SPA	Single Page Application – aplicação web que carrega uma única página HTML e atualiza o conteúdo dinamicamente via JavaScript.
JWT	JSON Web Token – padrão de autenticação stateless via token assinado digitalmente (RFC 7519).
REST	Representational State Transfer – estilo arquitetural para APIs HTTP sem estado entre requisições.
MVC	Model-View-Controller – padrão arquitetural que separa dados, lógica de negócio e apresentação.
ORM	Object-Relational Mapping – mapeamento automático entre objetos Java e tabelas relacionais.

Abreviação / Termo	Definição
WebSocket	Protocolo de comunicação bidirecional persistente sobre TCP para comunicação em tempo real.
STOMP	Simple Text Oriented Messaging Protocol – protocolo de mensageria sobre WebSocket usado com Spring.
QoS	Quality of Service – conjunto de qualidades que descrevem os requisitos não-funcionais de um sistema.
MVP	Minimum Viable Product – versão mínima viável com as funcionalidades essenciais para lançamento.
CI/CD	Continuous Integration / Continuous Delivery – pipeline automatizado de build, testes e entrega.
GM / Mestre	Game Master – condutor da narrativa em uma sessão de RPG de mesa.
JSONB	Tipo de dado do PostgreSQL que armazena JSON de forma binária com suporte a índices GIN.

1.4 Referências

- [KRU41]: The "4+1" view model of software architecture, Philippe Kruchten, IEEE Software, November 1995.
- [QOS]: Oracle – Quality of Service. https://docs.oracle.com/cd/E19636-01/819-2326/6n4kfe7dj/index.html
- Documento de Visão – RollCore / RPG Skill Checker & Companion – Equipe 9, v1.0, Mar/2026.
- Especificação de Casos de Uso (UC-01, UC-02, UC-03) – Equipe 9, v1.0, Mar/2026.
- Código-fonte do protótipo frontend (RollCore) – Equipe 9, Sprint 1 (Vercel).
- D&D 5e System Reference Document (SRD) – Wizards of the Coast / Creative Commons CC BY 4.0.
- Template DataSUS – Documento de Arquitetura de Software, Ministério da Saúde / DATASUS.
- Template GitHub – SysArq Documento de Arquitetura, FGA-EPS-MDS, 2021-2.

2. REPRESENTAÇÃO ARQUITETURAL

Este documento detalha as visões baseadas no modelo "4+1" [KRU41]. As visões utilizadas são:

Visão	Público-Alvo	Área de Interesse
Casos de Uso	Todos os stakeholders	Requisitos funcionais e comportamento do sistema
Lógica	Analistas e Arquitetos	Realização dos Casos de Uso, pacotes e camadas
Implementação	Programadores	Componentes de software, estrutura de código e dados
Implantação	Gerência de Configuração e DevOps	Nodos físicos, containers e infraestrutura
Processo	Integradores e Arquitetos	Performance, escalabilidade e concorrência

2.1 Diagrama de Relações (Visão Geral)

A arquitetura do RollCore adota o estilo em Camadas (Layered Architecture / N-Tier), com frontend e backend desacoplados e comunicação via API REST e WebSocket. O diagrama de relações entre os principais componentes é apresentado a seguir:

[CAMADA DE APRESENTAÇÃO]

React 18 + TypeScript (Web SPA) React Native + Expo (Mobile – Fase 2)

Zustand (estado global) | Axios (REST) | STOMP.js (WebSocket – Fase 2)

↓↑ **HTTPS (REST/JSON) + WSS/STOMP (Fase 2)** ↓↑

[CAMADA DE APLICAÇÃO / API]

Spring Boot 3 / Java 21 – REST Controllers | Spring Security + JWT | Rate Limiting

Engine D&D 5e (Java) | Dice Service (SecureRandom) | WebSocket Controller (Fase 2)

↓↑ **JDBC/JPA** ↓↑ ↓↑ **Redis Client** ↓↑

[CAMADA DE PERSISTÊNCIA]

PostgreSQL 16 (users, characters, dice_rolls, sessions) Redis 7 (sessões ao vivo, refresh tokens)

[INFRAESTRUTURA: Docker + Docker Compose | GitHub Actions CI/CD | Vercel (frontend) | AWS / Railway (backend)]

Figura 1 – Diagrama de Relações da Arquitetura do RollCore

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descreve os requisitos de software e restrições que têm impacto significativo na arquitetura. (Ref.: Seção 6 e 7 do Documento de Visão – Equipe 9.)

Linguagem	Backend: Java 21 LTS. Frontend Web: TypeScript 5.x. Frontend Mobile (Fase 2): TypeScript via React Native.
Framework	Backend: Spring Boot 3 (Spring MVC, Spring Security, Spring Data JPA, Spring WebSocket). Frontend: React 18 + Vite 5. Mobile: React Native + Expo SDK 50+.
Plataforma	Servidor de aplicação: Tomcat embarcado no Spring Boot, containerizado via Docker. Hospedagem: AWS EC2 ou Railway (backend), Vercel (frontend).
Segurança	Autenticação stateless via JWT (access token 15 min + refresh token 7 dias armazenado no Redis). Senhas hasheadas com BCrypt fator 12. HTTPS obrigatório em todas as comunicações. Rate limiting: 60 requisições/min por IP. CORS configurado no Spring Security.
Persistência	Banco relacional PostgreSQL 16 via Spring Data JPA / Hibernate (ORM). Cache in-memory e estado de sessões ao vivo via Redis 7. Campos de atributos de personagem em JSONB para extensibilidade entre sistemas de RPG.
Internacionalização (i18n)	A aplicação é desenvolvida integralmente em Português do Brasil (pt-BR), idioma dos usuários-alvo (jogadores e mestres de RPG lusófonos). Não há previsão de i18n para outros idiomas na Fase 1.
Escalabilidade	Arquitetura stateless no backend (JWT) permite múltiplas instâncias horizontais sem reconfiguração. Estado de sessões centralizado no Redis. Containerização via Docker facilita escalonamento por orquestrador.
Manutenibilidade	Cobertura de testes unitários ≥ 80% verificada via JaCoCo integrado ao GitHub Actions CI/CD. API documentada automaticamente via Springdoc / OpenAPI 3.0 (Swagger UI).
Desempenho	Tempo de resposta da API (P95) < 300ms com até 500 usuários simultâneos (RNF-01). Carregamento inicial da SPA < 3s em conexão 4G. Latência WebSocket em sessão < 1s (Fase 2).
Portabilidade	Interface web responsiva compatível com Chrome 110+, Firefox 110+, Safari 16+. Mobile via React Native (iOS 14+ e Android 10+) – Fase 2.

Tabela 1 – Requisitos e restrições arquiteturais

4. VISÃO DE CASOS DE USO

Esta seção lista os casos de uso centrais e significantes para a arquitetura do sistema RollCore na Fase 1 (MVP). Os casos de uso completos estão especificados no documento "RPG Skill Checker & Companion – Especificação de Casos de Uso, Equipe 9, v1.0, Mar/2026."

Lista de casos de uso do sistema:

- UC-01 – Cadastrar e Autenticar Usuário
- UC-02 – Criar e Gerenciar Ficha de Personagem D&D 5e
- UC-03 – Rolar Dados Virtuais
- UC-04 – Gerenciar Sessão Online (Fase 2)

4.1 Casos de Uso Significantes para a Arquitetura

Os três casos de uso da Fase 1 (UC-01, UC-02, UC-03) são todos críticos para a arquitetura, pois estabelecem os três pilares funcionais do sistema:

Caso de Uso	Ator(es)	Impacto Arquitetural
UC-01 – Cadastrar e Autenticar Usuário	Jogador, Mestre (GM)	Define a camada de segurança completa: Spring Security, JWT (access + refresh), BCrypt, rate limiting, tabela users no PostgreSQL, refresh tokens no Redis.
UC-02 – Criar e Gerenciar Ficha D&D 5e	Jogador	Aciona a engine de regras D&D 5e (calcMod, profBonus, slots de magia) tanto no frontend (lib/engine.ts) quanto no backend (módulo Java). Define o uso de JSONB no campo attributes da tabela characters.
UC-03 – Rolar Dados Virtuais	Jogador, Mestre (GM)	Aciona o Dice Service com java.security.SecureRandom server-side. Define a tabela dice_rolls com JSONB para individual_results. Na Fase 2, aciona o WebSocket Controller para transmissão em tempo real via STOMP.
UC-04 – Gerenciar Sessão (Fase 2)	Mestre (GM), Sistema	Define o Serviço de Sessões com Spring WebSocket + STOMP, estado em Redis (chave session:{code}), e o tópico /topic/session.{codigo} para broadcast de rolagens.

Tabela 2 – Casos de uso significantes para a arquitetura

O diagrama a seguir representa os casos de uso gerais do sistema e os atores envolvidos:

Diagrama de Casos de Uso – RollCore (Fase 1)

Jogador
|► UC-01 Cadastrar/Autenticar ► UC-02 Criar Ficha ► UC-03 Rolar Dados |

Mestre (GM)
|► UC-01 Autenticar ► UC-03 Rolar Dados ► UC-04 Gerenciar Sessão (Fase 2) |

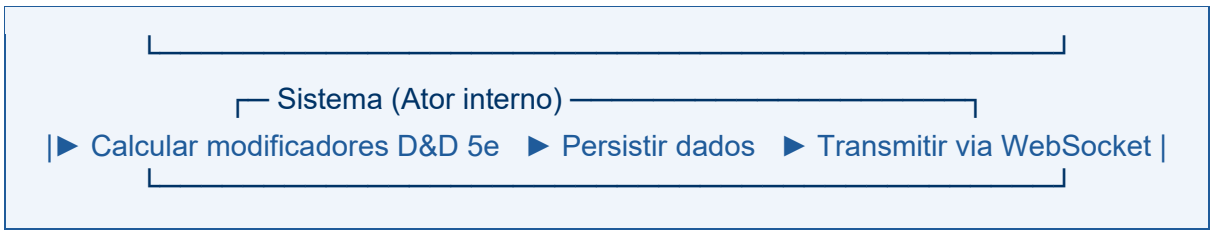

Figura 2 – Diagrama com os casos de uso significativos e atores

5. VISÃO LÓGICA

Esta seção descreve a visão lógica da arquitetura: as principais classes e módulos, sua organização em pacotes, camadas e subsistemas, bem como aspectos dinâmicos representados por diagramas de sequência.

5.1 Visão Geral – Pacotes e Camadas

A arquitetura do RollCore é organizada em três grandes camadas, cada uma com responsabilidades bem definidas:

Camada	Pacote / Módulo	Componentes Internos	Responsabilidade
Apresentação (Web)	src/ (React SPA)	App.tsx, screens/, store/, lib/, types/	UI, navegação, estado global (Zustand), chamadas à API REST.
Aplicação (Backend)	br.com.rollcore.*	controller/, service/, security/, websocket/	Exposição da API REST, autenticação, lógica de negócio, engine D&D 5e, Dice Service.
Persistência	br.com.rollcore.repository	repository/, entity/, PostgreSQL, Redis	ORM via JPA, acesso ao banco relacional e ao cache Redis.

Tabela 3 – Visão geral de camadas e pacotes

5.1.1 Camada de Apresentação (Frontend)

A camada de apresentação é uma SPA desenvolvida em React 18 + TypeScript, organizada nos seguintes pacotes principais:

Pacote / Diretório	Componentes	Padrão / Responsabilidade
src/components/auth/	LoginScreen, RegisterScreen, ForgotPasswordScreen, ProfileScreen	Telas de autenticação e perfil. (UC-01)
src/components/characters/	CharacterListScreen, CharacterFormScreen, CharacterSheetScreen	Gestão de fichas D&D 5e com cálculo em tempo real. (UC-02)
src/components/dice/	DiceRollerScreen	Rolagem de dados com fórmulas NdX+M e histórico. (UC-03)
src/components/ui/	Toast, Modal, DiceLogo	Componentes reutilizáveis de UI.
src/store/	useAppStore.ts (Zustand)	Estado global: auth, personagens, histórico, navegação, toasts.
src/lib/	engine.ts, dice.ts, storage.ts	Engine D&D 5e (calcMod, profBonus, skills), parser de dados, persistência local.
src/types/	index.ts	Interfaces TypeScript de domínio: Character, RollResult, User, Screen, HistoryEntry.

5.1.2 Camada de Aplicação (Backend Spring Boot)

O backend é estruturado no padrão MVC do Spring Boot com os seguintes pacotes:

Pacote	Componentes	Responsabilidade
controller	AuthController, CharacterController, DiceController, WebSocketController (Fase 2)	Endpoints REST e WebSocket. Recebe requisições, valida JWT e delega para Services.
service	AuthService, CharacterService, DiceService, SessionService (Fase 2)	Lógica de negócio: JWT, BCrypt, engine D&D 5e, SecureRandom, WebSocket broadcast.
security	JwtFilter, SecurityConfig, RateLimitingFilter	Cadeia de filtros: validação de token JWT, rate limiting 60 req/min/IP, configuração CORS.
engine	DnD5eEngine (calcMod, profBonus, skillBonus, slots de magia)	Módulo Java puro com regras D&D 5e conforme SRD. Sem dependências de framework.
repository	UserRepository, CharacterRepository, DiceRollRepository, SessionRepository	Spring Data JPA. Queries JPQL e suporte a JSONB via Hibernate Types.
entity	User, Character, DiceRoll, Session	Entidades JPA mapeadas para as tabelas PostgreSQL.

5.1.3 Diagrama de Camadas

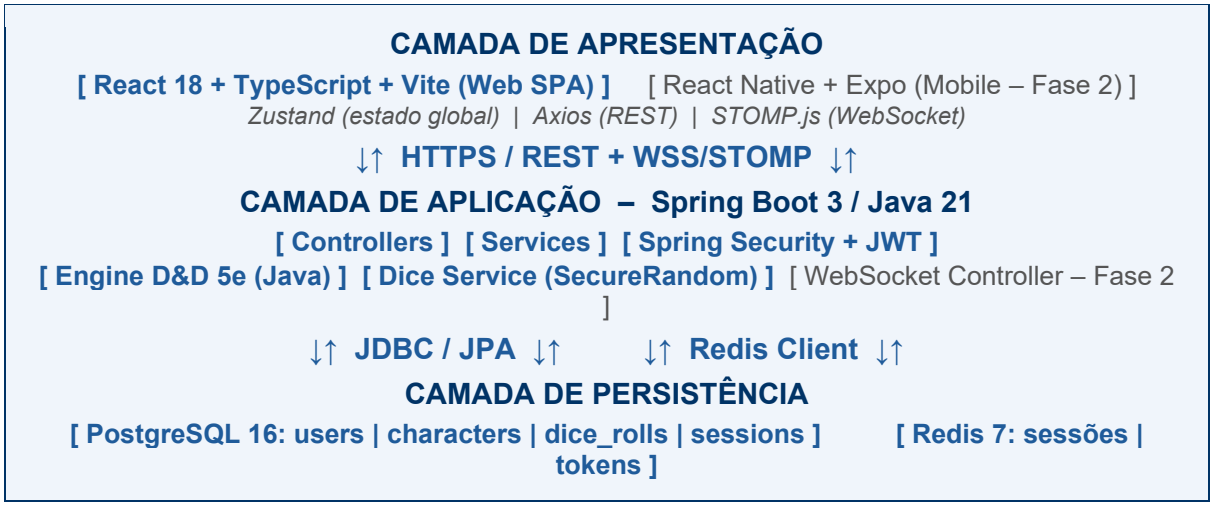

Figura 3 – Diagrama de Camadas da Aplicação RollCore

6. VISÃO DE IMPLEMENTAÇÃO

Esta seção apresenta os diagramas de implementação dos casos de uso mais significantes para a arquitetura, incluindo o diagrama de sequência e o modelo de dados (Entidade-Relacionamento e Lógico de Dados).

6.1 UC-01 – Cadastrar e Autenticar Usuário

6.1.1 Diagrama de Sequência – Login

Diagrama de Sequência – UC-01 (Login)

Cliente (React) → POST /auth/login {email, senha} → **AuthController**
AuthController → userService.authenticate(email, senha) → **UserService**
UserService → findByEmail(email) → **UserRepository / PostgreSQL**
UserService ← user (com password_hash) ← **UserRepository**
UserService → BCrypt.verify(senha, password_hash) → **BCrypt**
UserService ← true / false ← **BCrypt**
UserService → jwtService.generateAccessToken(userId) → **JwtService**
UserService → redisService.storeRefreshToken(userId, token, 7d) → **Redis 7**
AuthController ← {accessToken, refreshToken} ← **UserService**
Cliente ← 200 OK {accessToken (15min), refreshToken (7d)} ← **AuthController**

Figura 4 – Diagrama de Sequência UC-01 (Login)

6.2 UC-03 – Rolar Dados Virtuais

6.2.1 Diagrama de Sequência – Rolar Dados

Diagrama de Sequência – UC-03 (Rolar Dados)

Cliente (React) → POST /dice/roll {formula: "2d6+3"} + JWT → **DiceController**
DiceController → jwtFilter.validateToken(JWT) → **JwtFilter / Spring Security**
DiceController → diceService.roll("2d6+3") → **DiceService**
DiceService → parse("2d6+3") → n=2, sides=6, mod=3 → **DiceParser**
DiceService → SecureRandom.nextInt(6)+1 ×2 → **java.security.SecureRandom**
DiceService ← [4, 2] → total = 4+2+3 = 9 ← **SecureRandom**
DiceService → diceRollRepo.save(DiceRoll{...}) → **DiceRollRepository / PostgreSQL**
DiceController ← {formula, rolls:[4,2], mod:3, total:9} ← **DiceService**
Cliente ← 200 OK {resultado} (exibe animação + histórico) ← **DiceController**

Figura 5 – Diagrama de Sequência UC-03 (Rolar Dados)

6.3 Diagrama Entidade-Relacionamento (DER)

Diagrama Entidade-Relacionamento – RollCore (Fase 1)

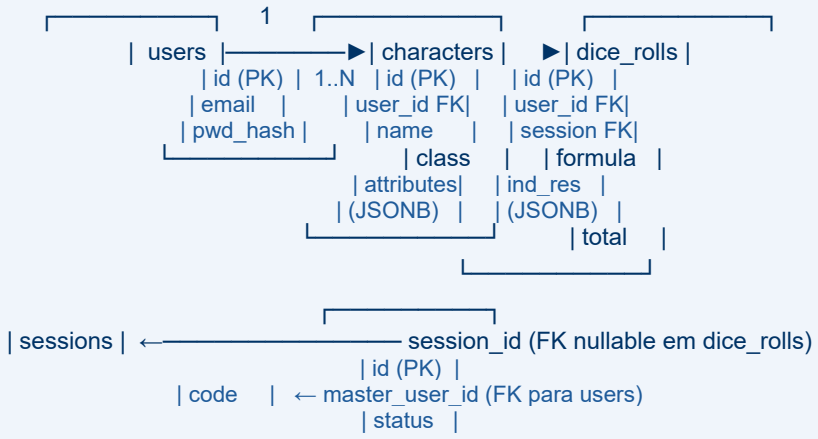

Figura 6 – Diagrama Entidade-Relacionamento (DER) – RollCore

6.4 Diagrama Lógico de Dados

Tabela	Coluna	Tipo	PK/FK	Obrig.	Descrição
users	id	UUID	PK	Sim	Identificador único do usuário.
users	email	VARCHAR(255)	-	Sim	E-mail de login, único no sistema (UNIQUE constraint).
users	password_hash	VARCHAR(255)	-	Sim	Hash BCrypt (fator 12) da senha do usuário.
users	created_at	TIMESTAMP	-	Sim	Data e hora de criação do registro.
users	updated_at	TIMESTAMP	-	Sim	Data e hora da última atualização.
characters	id	UUID	PK	Sim	Identificador único do personagem.
characters	user_id	UUID	FK→users	Sim	Referência ao usuário dono da ficha.
characters	name	VARCHAR(100)	-	Sim	Nome do personagem.
characters	class	VARCHAR(50)	-	Sim	Classe D&D (Guerreiro, Mago, Ladino, etc.).
characters	subclass	VARCHAR(50)	-	Não	Subclasse escolhida (vazio até o nível requerido).
characters	race	VARCHAR(50)	-	Sim	Raça do personagem (Humano, Elfo, Anão, etc.).
characters	level	INT	-	Sim	Nível atual do personagem (1–20).
characters	attributes	JSONB	-	Sim	Atributos: {STR, DEX, CON, INT, WIS, CHA}.

Tabela	Coluna	Tipo	PK/FK	Obrig.	Descrição
					Extensível via JSONB.
characters	hp	INT	-	Sim	Pontos de vida atuais.
characters	max_hp	INT	-	Sim	Pontos de vida máximos.
characters	ac	INT	-	Sim	Classe de Armadura calculada.
dice_rolls	id	UUID	PK	Sim	Identificador único da rolagem.
dice_rolls	user_id	UUID	FK→users	Sim	Referência ao usuário que realizou a rolagem.
dice_rolls	session_id	UUID	FK→sessions	Não	NULL se fora de sessão; ID da sessão se em sala ativa.
dice_rolls	formula	VARCHAR(50)	-	Sim	Fórmula da rolagem (ex: 2d6+3).
dice_rolls	individual_results	JSONB	-	Sim	Array de resultados individuais por dado.
dice_rolls	total	INT	-	Sim	Resultado total calculado.
dice_rolls	rolled_at	TIMESTAMP	-	Sim	Data e hora da rolagem.
sessions	id	UUID	PK	Sim	Identificador único da sessão.
sessions	code	VARCHAR(10)	-	Sim	Código de acesso único da sala (UNIQUE constraint).
sessions	master_user_id	UUID	FK→users	Sim	Referência ao Mestre (GM) da sessão.
sessions	name	VARCHAR(100)	-	Sim	Nome descritivo da sessão.
sessions	status	VARCHAR(20)	-	Sim	Status da sessão: ACTIVE ou CLOSED.

Tabela	Coluna	Tipo	PK/FK	Obrig.	Descrição
sessions	created_at	TIMESTAMP	-	Sim	Data e hora de criação da sessão.

Tabela 4 – Diagrama Lógico de Dados (DLD) – RollCore

7. VISÃO DE IMPLANTAÇÃO

Esta seção descreve os nodos físicos, configurações e artefatos que serão implantados em produção.

Nodo	Tipo	Artefatos Implantados	Tecnologia de Deploy
Servidor de Aplicação	VPS Cloud (AWS EC2 t3.small ou Railway)	rollcore-backend.jar (Spring Boot Fat JAR), containerizado	Docker Container + Docker Compose
Servidor de Banco de Dados	Serviço gerenciado ou container Docker	PostgreSQL 16 (schema: users, characters, dice_rolls, sessions)	Docker Container ou Supabase (free tier)
Servidor de Cache	Container Docker ou Redis Cloud	Redis 7 (sessões ao vivo, refresh tokens com TTL)	Docker Container ou Redis Cloud (free tier)
Servidor de Frontend	CDN Vercel (já configurado)	Bundle React/Vite (HTML + JS + CSS minificados)	Vercel CI/CD (Git push → deploy automático)
Pipeline CI/CD	GitHub Actions (nuvem)	Build JAR, testes JaCoCo, build Docker image, push registry, deploy	GitHub Actions Workflows
Certificado SSL	Let's Encrypt (gratuito)	Certificado TLS para HTTPS obrigatório	Certbot / Vercel automático

Tabela 5 – Nodos físicos e artefatos de implantação

Diagrama de Implantação – RollCore

[Usuário: Navegador Chrome / Firefox / Safari / App Mobile]

↓ HTTPS (443) ↑

[Vercel CDN: React SPA (HTML/JS/CSS) – deploy automático via GitHub]

[AWS EC2 / Railway: Docker – Spring Boot JAR + PostgreSQL 16 + Redis 7]

↓ JDBC/JPA | Redis Client ↑

[GitHub Actions CI/CD: build → JaCoCo (≥80%) → Docker image → deploy]

Figura 7 – Diagrama de Implantação do RollCore

8. DIMENSIONAMENTO E PERFORMANCE

O RollCore é uma aplicação web voltada a jogadores e mestres de RPG de mesa lusófonos. O sistema é projetado para crescimento gradual, iniciando com o MVP (Fase 1) e expandindo para sessões online em tempo real (Fase 2).

8.1 Volume

Estimativas de volume de acesso para o MVP (Fase 1):

Número estimado de usuários (MVP)	500 usuários cadastrados nos primeiros 3 meses após lançamento.
Número estimado de usuários (Fase 2)	2.000–5.000 usuários ativos mensais (grupos de RPG online).
Acessos diários estimados	200–500 requisições REST/dia no MVP; 2.000–10.000 na Fase 2 com sessões online.
Acessos por período de pico	Fins de semana (sessões de RPG típicas): 60–80% do tráfego semanal concentrado entre sexta e domingo.
Tempo de sessão por usuário	20–90 minutos por sessão de jogo (Fase 2 com WebSocket ativo).
Volume de personagens por usuário	Média de 3–5 fichas de personagem por usuário ativo.
Volume de rolagens de dados	50–200 rolagens por sessão de jogo ativa (UC-03).
Tamanho estimado do banco (MVP)	< 500 MB para 500 usuários com até 5 personagens e histórico de rolagens.

Tabela 6 – Estimativas de volume

8.2 Performance

Metas de performance do sistema, conforme Documento de Visão – Equipe 9 (RNF-01 a RNF-03):

Tempo de resposta da API REST (P95)	< 300ms com até 500 usuários simultâneos. Garantido via pool HikariCP, índices PostgreSQL e Java 21 virtual threads.
Tempo de carregamento inicial da SPA	< 3 segundos em conexão 4G. Garantido via build otimizado pelo Vite com code splitting e CDN Vercel.
Latência WebSocket em sessão (Fase 2)	< 1 segundo (P95) entre rolagem e recebimento por todos os participantes da sala, em rede 4G com latência ≤ 150ms.
Disponibilidade (uptime)	≥ 99,5% mensal. Deploy containerizado com política de reinicialização automática (restart: always no Docker Compose).
Concorrência máxima suportada (MVP)	500 usuários simultâneos com degradação aceitável. Escalonamento horizontal disponível via Docker sem reconfiguração.
Tempo máximo para login (JWT)	< 200ms incluindo verificação BCrypt (fator 12) e geração do token JWT.

Tabela 7 – Metas de performance

9. QUALIDADE

Esta seção enumera os itens de qualidade de software [QOS] significativos para a aplicação RollCore, com suas descrições e as soluções arquiteturais adotadas para atendê-los.

Item de Qualidade	Descrição	Solução Arquitetural
Escalabilidade	Capacidade de crescer horizontalmente para atender aumento de usuários sem refatoração total da arquitetura.	Backend stateless com JWT (sem sessão server-side) permite múltiplas instâncias. Redis centraliza estado de sessões ao vivo. Docker facilita orquestração com múltiplos containers.
Confiabilidade e Disponibilidade	O sistema deve estar disponível com uptime ≥ 99,5% mensal, sem perda de dados em falhas.	Deploy em VPS Cloud com reinicialização automática (Docker restart policy). Transações ACID no PostgreSQL garantem integridade. Refresh token armazenado no Redis com TTL.
Segurança	Proteção de dados dos usuários, prevenção de acesso não autorizado e integridade das rolagens de dados.	Spring Security + JWT (access 15min / refresh 7d). BCrypt fator 12 para senhas. HTTPS obrigatório. Rate limiting 60 req/min/IP. java.security.SecureRandom server-side para rolagens (previne manipulação client-side).
Portabilidade	Suporte a múltiplos navegadores web e, futuramente, dispositivos móveis iOS e Android.	Frontend em React 18 (Chrome 110+, Firefox 110+, Safari 16+). Futura versão mobile via React Native + Expo, reutilizando a mesma API REST sem duplicação de backend.
Manutenibilidade	Facilidade de alterar, testar e evoluir o sistema ao longo das fases do roadmap.	Cobertura de testes unitários ≥ 80% verificada por JaCoCo integrado ao GitHub Actions CI/CD (bloqueia merge se regredir). API documentada automaticamente via Springdoc / OpenAPI 3.0. Separação em pacotes por responsabilidade (controller/service/repository/engine).
Extensibilidade	Capacidade de adicionar novos sistemas de RPG (ex: Ordem Paranormal – Fase 3) sem refatoração do schema.	Campo attributes em JSONB no PostgreSQL permite adicionar diferentes conjuntos de atributos por sistema de RPG sem alterar o schema relacional. Módulo engine D&D 5e desacoplado em Java puro, substituível por outros engines futuros.
Integridade das Rolagens	Garantia de que os resultados de dados não possam ser manipulados pelo cliente.	Processamento de rolagens exclusivamente no servidor com java.security.SecureRandom (geração criptograficamente segura). Cliente apenas envia a fórmula; servidor processa e retorna o resultado.
Rastreabilidade	Capacidade de auditar quem fez o quê e quando no sistema.	Tabela dice_rolls registra todas as rolagens com user_id, session_id, formula, resultados e timestamp. Logs de autenticação via Spring Security. Histórico limitado a 50 rolagens na interface, mas completo no banco para auditoria.

Tabela 8 – Itens de qualidade de software (QoS)

10. REFERÊNCIAS

- [KRU41]: KRUCHTEN, Philippe. The "4+1" view model of software architecture. IEEE Software, v.12, n.6, p.42-50, November 1995.
- [QOS]: Oracle – Quality of Service. Disponível em: https://docs.oracle.com/cd/E19636-01/819-2326/6n4kfe7dj/index.html
- Documento de Visão – RPG Skill Checker & Companion – Equipe 9, v1.0, Mar/2026.
- Especificação de Casos de Uso (UC-01, UC-02, UC-03) – Equipe 9, v1.0, Mar/2026.
- Código-fonte do protótipo frontend (RollCore.zip) – Equipe 9, Sprint 1. Implantado em: https://vercel.com.
- D&D 5e System Reference Document (SRD) – Wizards of the Coast / Creative Commons CC BY 4.0. Disponível em: https://dnd.wizards.com/resources/systems-reference-document.
- Template DataSUS – Documento de Arquitetura de Software. Ministério da Saúde / DATASUS, MDS Versão 1.0.
- Template GitHub – SysArq Documento de Arquitetura. FGA-EPS-MDS, 2021-2. Disponível em: https://fga-eps-mds.github.io/2021-2-SysArq-Doc/doc_arquitetura/
- Spring Boot 3 Reference Documentation. Disponível em: https://docs.spring.io/spring-boot/docs/3.x/reference/html/
- React 18 Documentation. Disponível em: https://react.dev
- PostgreSQL 16 Documentation. Disponível em: https://www.postgresql.org/docs/16/
- Redis 7 Documentation. Disponível em: https://redis.io/docs/

Metodologia de Desenvolvimento de Software – RollCore – Equipe 9 – Versão 1.0